AIR UNIVERSITY AEROSPACE AND AVIATION CAMPUS KAMRA

SUBMITTED TO: SIR GHULAM ALI

SUBMITTED BY: NOOR-UL-AIN

REGISTRATION ID: 235135

COURSE: DIP LAB

PROGRAM: BS CS

SEMESTER: 06

DATE: 27th March, 2026

LAB TASK # 6

1. Objective:

The primary objective of this project is to develop a professional, integrated GUI application using **Python (OpenCV and Matplotlib)**. The system aims to demonstrate core DIP concepts—including metadata extraction, sampling, quantization, geometric/intensity transformations, and automated enhancement—within a single-window environment.

2. Methodology:

Phase 1: Acquisition & UI Design:

Implementing a **CustomTkinter** sidebar navigation and tabview system. This phase focuses on reading images as numerical matrices and displaying metadata like resolution and data type.

Phase 2: Sampling & Quantization:

Analyzing image degradation by reducing spatial resolution (Sampling) and color depth (Quantization).

Phase 3: Transformation Engines:

Building the backend logic for rotating and shearing images (Geometric) and applying mathematical transforms like Log and Gamma (Intensity).

Phase 4: Statistical Analysis:

Using **Matplotlib** to generate and compare histograms to visualize pixel distribution before and after equalization.

Phase 5: Smart Pipeline Integration:

Designing an automated sequence that combines Gamma Correction and Histogram Equalization for optimal visual output.

3. Code Snippets:

The Integrated Display Logic:

 # Helper to show OpenCV images inside CustomTkinter tabs

```
def update_display(self, cv_img, label_widget):
    rgb = cv2.cvtColor(cv_img, cv2.COLOR_GRAY2RGB)
    pil_img = Image.fromarray(rgb)
    tk_img = ImageTk.PhotoImage(pil_img)
    label_widget.configure(image=tk_img)
```

The Smart Enhancement Pipeline:

```
# Automated Gamma + Histogram Equalization
lut = np.array([((i/255.0)**(1.0/1.6))*255 for i in np.arange(0,256)]).astype("uint8")
self.final_result = cv2.equalizeHist(cv2.LUT(self.gray_img, lut))
```

4. Input Images:

5. Output Images:

6. Observation and Analysis:

Sampling Observation:

Reducing the scale to **0.25x** introduced significant **Aliasing** (jagged edges) when resized back to original dimensions.

Quantization Observation:

Lowering bit-depth to **2-bit** caused **False Contouring**, where smooth gradients were replaced by distinct bands of gray.

Histogram Analysis:

Histogram Equalization effectively spread out the pixel intensity distribution, significantly improving the contrast in low-light test images.

7. Conclusion:

The project successfully meets all requirements for a professional DIP application. By using **Python's OpenCV** for high-speed matrix manipulation and **Matplotlib** for data visualization, we created a tool that not only transforms images but provides deep analytical insight into the underlying data. The final "Smart System" proves that combining simple intensity transforms with statistical equalization can yield high-quality automated results.